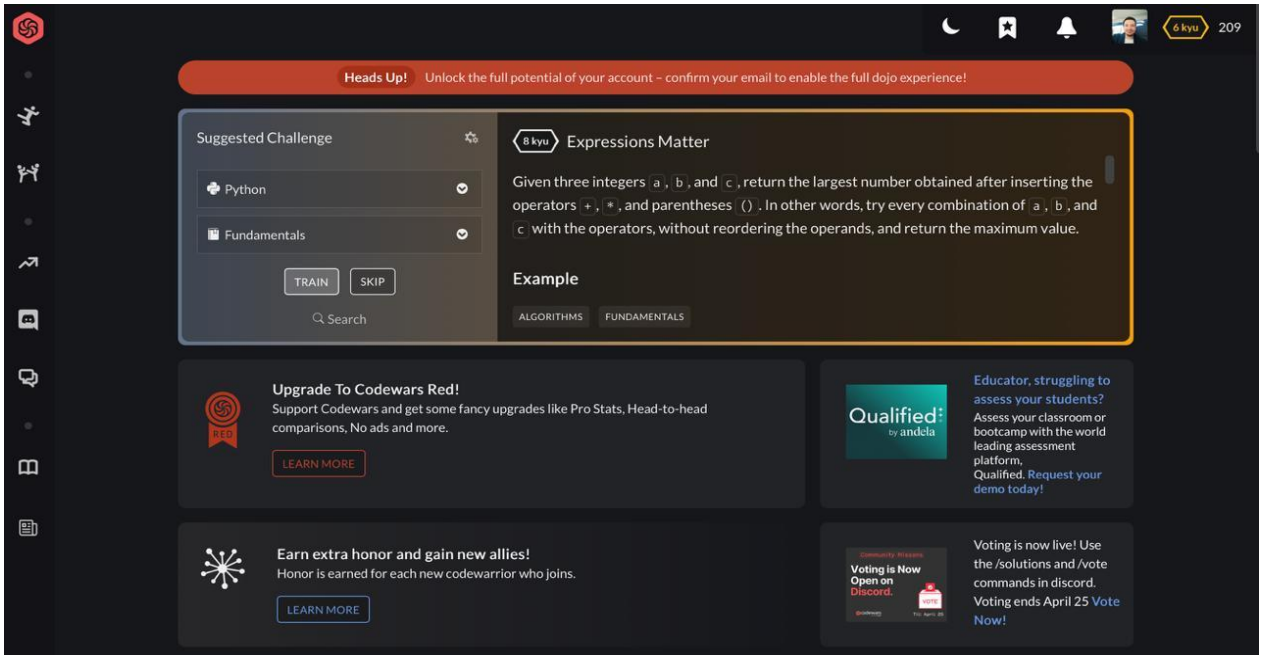
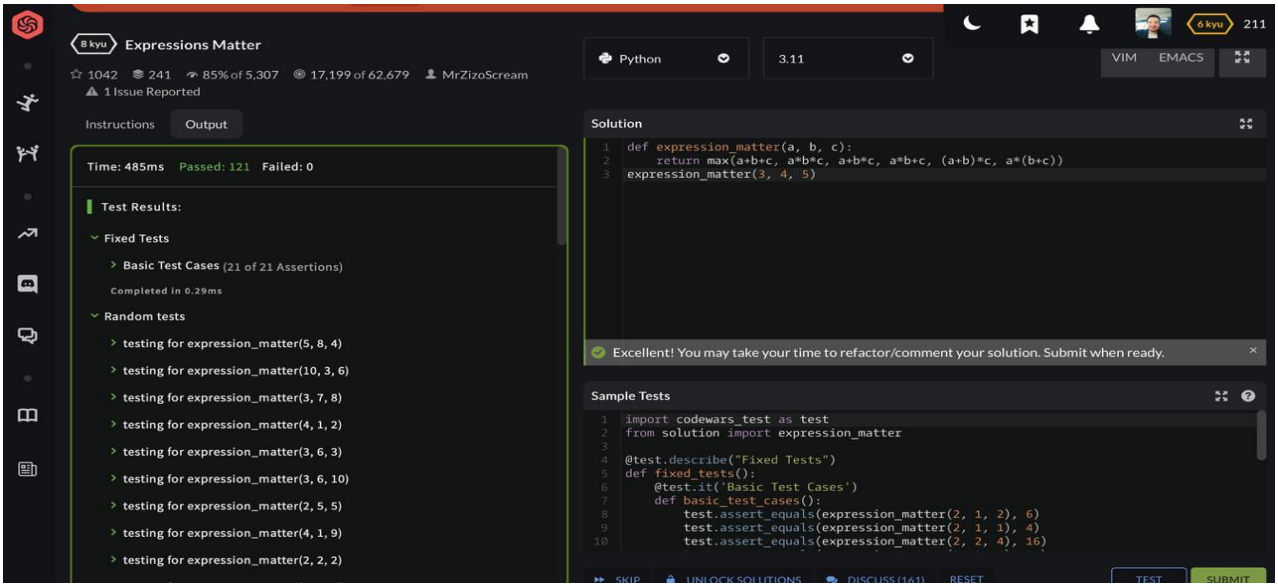
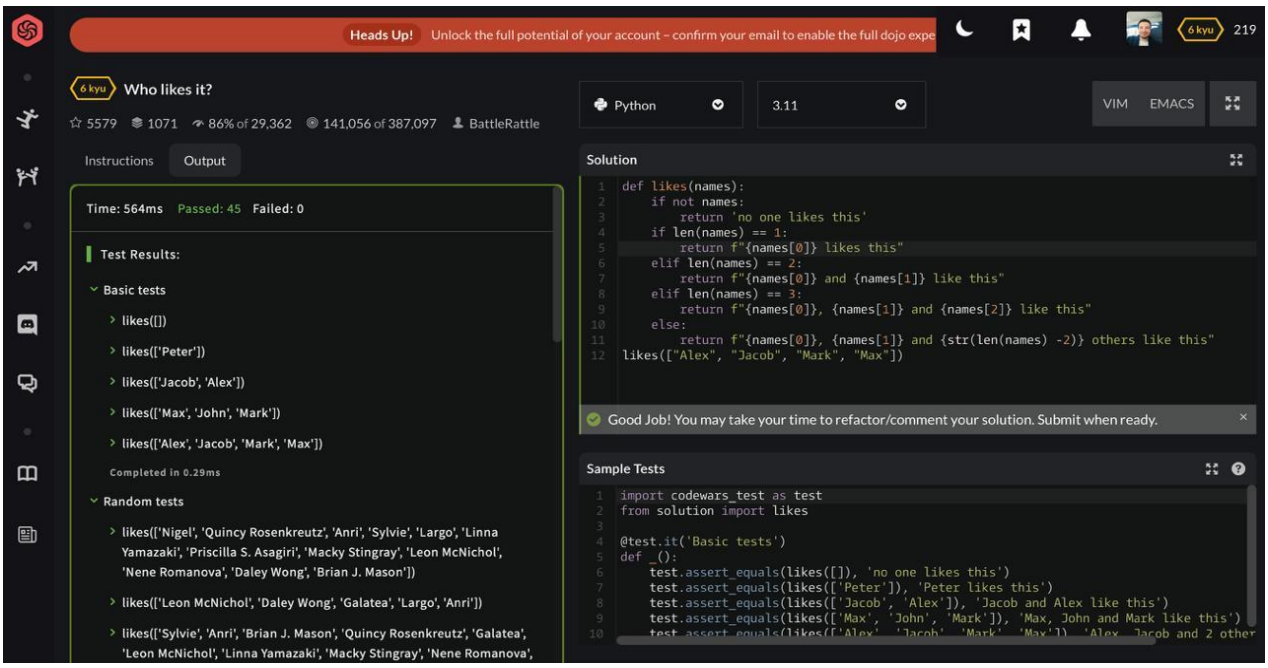
Avval


6 kyu 209
Heads Up! Unlock the full potential of your account – confirm your email to enable the full dojo experience!
Suggested Challenge
Python
Fundamentals
TRAIN
SKIP
Search
8 kyu Expressions Matter
Given three integers a, b, and c, return the largest number obtained after inserting the operators +, *, and parentheses (). In other words, try every combination of a, b, and c with the operators, without reordering the operands, and return the maximum value.
Example
ALGORITHMS
FUNDAMENTALS
Upgrade To Codewars Red!
Support Codewars and get some fancy upgrades like Pro Stats, Head-to-head comparisons, No ads and more.
LEARN MORE
Qualified by andela
Educator, struggling to assess your students?
Assess your classroom or bootcamp with the world leading assessment platform, Qualified. Request your demo today!
Earn extra honor and gain new allies!
Honor is earned for each new codewarrior who joins.
LEARN MORE
Voting is Now Open on Discord.
Voting is now live! Use the /solutions and /vote commands in discord. Voting ends April 25 Vote Now!

Keyin


6 kyu 211
8 kyu Expressions Matter
1042 241 85% of 5,307 17,199 of 62,679 MrZizoScream
1 Issue Reported
Instructions Output
Python 3.11
VIM EMACS
Time: 485ms Passed: 121 Failed: 0
Test Results:
Fixed Tests
Basic Test Cases (21 of 21 Assertions)
Completed in 0.29ms
Random tests
testing for expression_matter(5, 8, 4)
testing for expression_matter(10, 3, 6)
testing for expression_matter(3, 7, 8)
testing for expression_matter(4, 1, 2)
testing for expression_matter(3, 6, 3)
testing for expression_matter(3, 6, 10)
testing for expression_matter(2, 5, 5)
testing for expression_matter(4, 1, 9)
testing for expression_matter(2, 2, 2)
Solution
def expression_matter(a, b, c):
return max(a+b+c, a*b*c, a+b*c, a*b+c, (a+b)*c, a*(b+c))
expression_matter(3, 4, 5)
Excellent! You may take your time to refactor/comment your solution. Submit when ready.
Sample Tests
import codewars_test as test
from solution import expression_matter
@test.describe("Fixed Tests")
def fixed_tests():
@test.it('Basic Test Cases')
def basic_test_cases():
test.assert_equals(expression_matter(2, 1, 2), 6)
test.assert_equals(expression_matter(2, 1, 1), 4)
test.assert_equals(expression_matter(2, 2, 4), 16)
SKIP UNLOCK SOLUTIONS DISCUSS (161) RESET TEST SUBMIT


Heads Up! Unlock the full potential of your account – confirm your email to enable the full dojo expe
6 kyu 219
6 kyu Who likes it?
5579 1071 86% of 29,362 141,056 of 387,097 BattleRattle
Instructions Output
Python 3.11
VIM EMACS
Time: 564ms Passed: 45 Failed: 0
Test Results:
Basic tests
likes([])
likes(['Peter'])
likes(['Jacob', 'Alex'])
likes(['Max', 'John', 'Mark'])
likes(['Alex', 'Jacob', 'Mark', 'Max'])
Completed in 0.29ms
Random tests
likes(['Nigel', 'Quincy Rosenkreutz', 'Anri', 'Sylvie', 'Largo', 'Linna Yamazaki', 'Priscilla S. Asagiri', 'Macky Stingray', 'Leon McNichol', 'Nene Romanova', 'Daley Wong', 'Brian J. Mason'])
likes(['Leon McNichol', 'Daley Wong', 'Galatea', 'Largo', 'Anri'])
likes(['Sylvie', 'Anri', 'Brian J. Mason', 'Quincy Rosenkreutz', 'Galatea', 'Leon McNichol', 'Linna Yamazaki', 'Macky Stingray', 'Nene Romanova',
Solution
def likes(names):
if not names:
return 'no one likes this'
if len(names) == 1:
return f"{names[0]} likes this"
elif len(names) == 2:
return f"{names[0]} and {names[1]} like this"
elif len(names) == 3:
return f"{names[0]}, {names[1]} and {names[2]} like this"
else:
return f"{names[0]}, {names[1]} and {str(len(names) -2)} others like this"
likes(["Alex", "Jacob", "Mark", "Max"])
Good Job! You may take your time to refactor/comment your solution. Submit when ready.
Sample Tests
import codewars_test as test
from solution import likes
@test.it('Basic tests')
def _():
test.assert_equals(likes([]), 'no one likes this')
test.assert_equals(likes(['Peter']), 'Peter likes this')
test.assert_equals(likes(['Jacob', 'Alex']), 'Jacob and Alex like this')
test.assert_equals(likes(['Max', 'John', 'Mark']), 'Max, John and Mark like this')